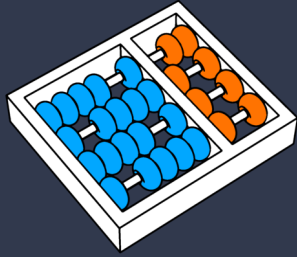




UNICAMP



Workflows e Orquestração Agêntica com LangGraph

Prof. Dr. Julio Cesar dos Reis

dosreis@unicamp.br

Agenda

- ❑ LLMs como componentes de workflow
- ❑ Estado conversacional
- ❑ Roteamento orientado por LLM
- ❑ Workflows multi-etapas
- ❑ Arquiteturas de agentes em grafos

Motivação

- ❑ Um LLM isolado não é um agente
- ❑ Um agente é um sistema composto por múltiplos componentes coordenados

Problema

- ❑ Conforme as tarefas aumentam de complexidade
 - ❑ os prompts crescem demais
 - ❑ a manutenção se torna difícil
 - ❑ o raciocínio fica pouco controlável
 - ❑ a reutilização desaparece

Evolução da Engenharia de IA

Existe uma **transição importante** na engenharia de IA



A evolução permite lidar com tarefas de complexidade crescente, garantindo controle, manutenibilidade e reutilização.

Exemplo

Objetivo: escrever um relatório.

Abordagem “Ingênua”

Prompt Único

"Pesquise, planeje, escreva, revise e entregue um relatório."

Uma única chamada ao modelo para realizar todas as etapas sequencialmente.

Abordagem Agêntica

Planejar



Pesquisar



Gerar

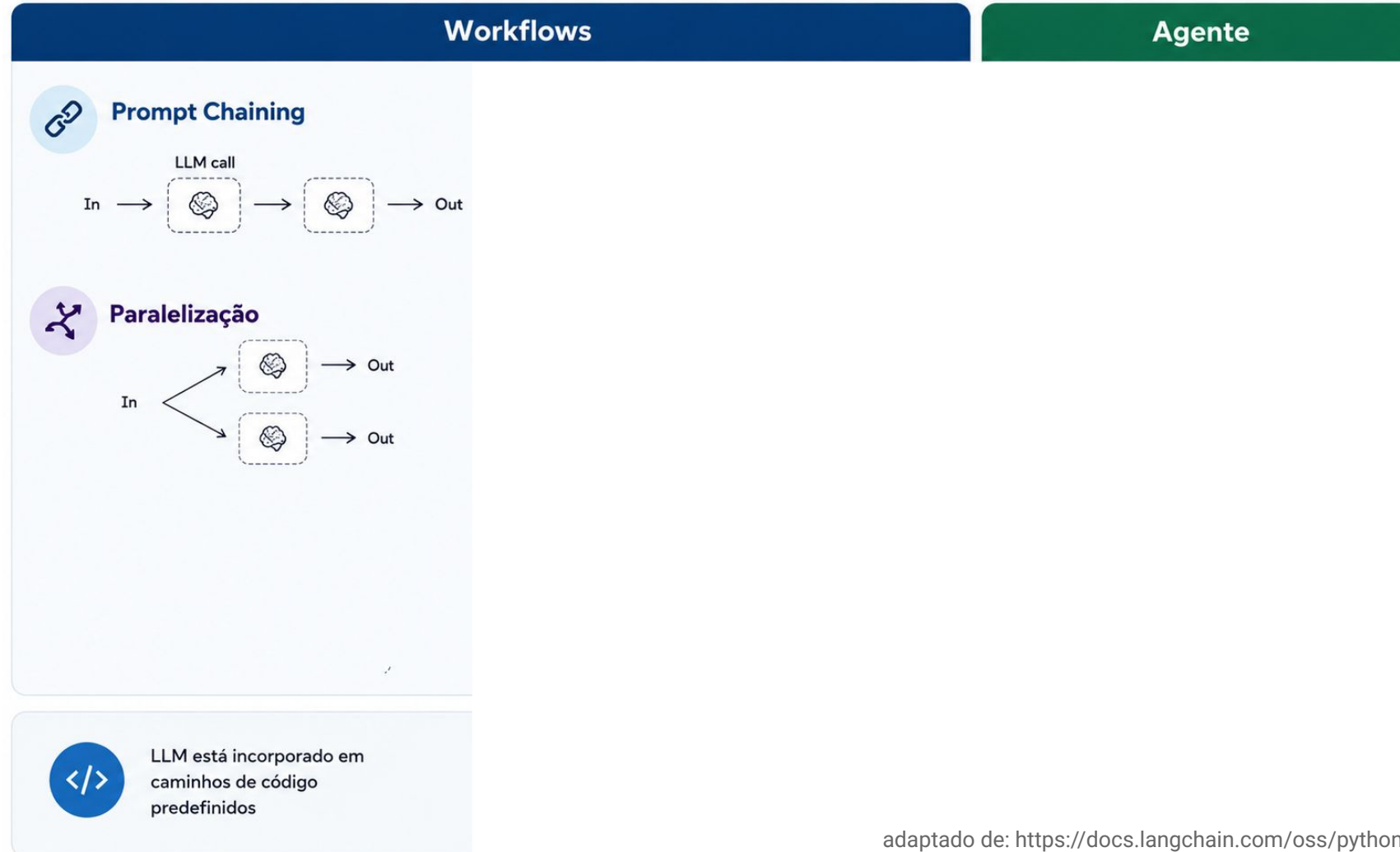


Revisar

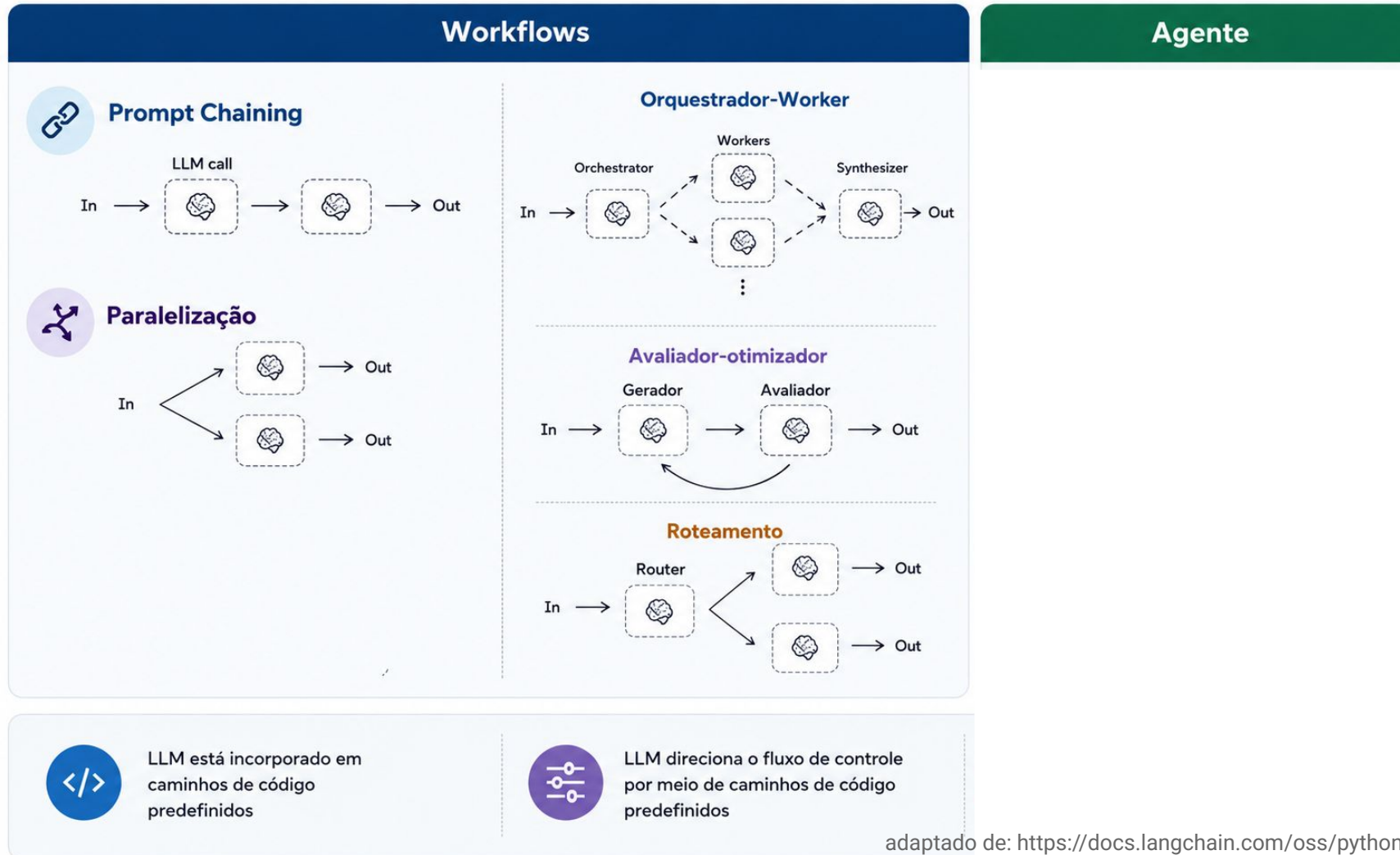


Entregar

Padrões de Workflow & Agente

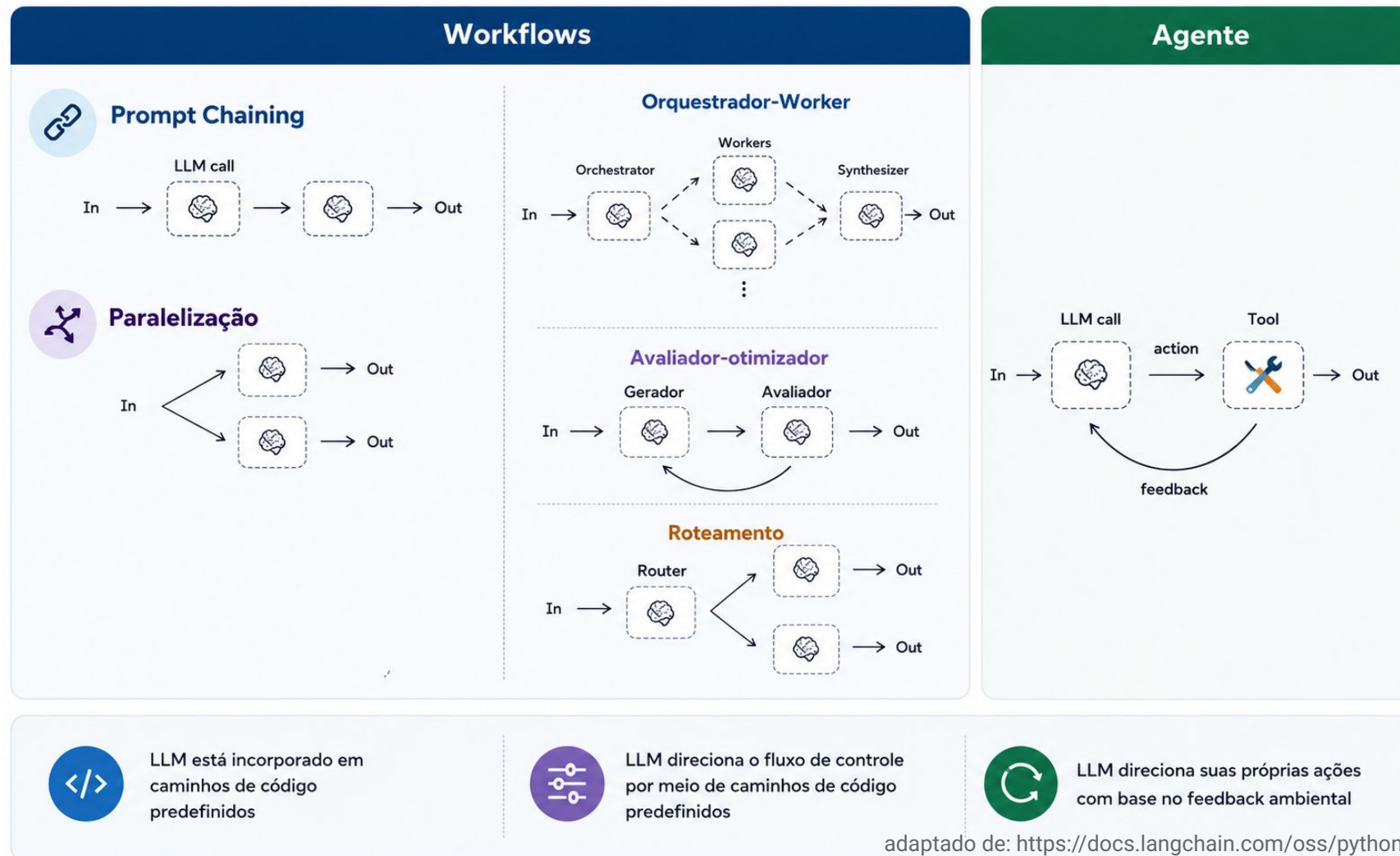


Padrões de Workflow & Agente



adaptado de: <https://docs.langchain.com/oss/python/langgraph/workflows-agents>

Padrões de Workflow & Agente



Discussão

- ❑ Todo problema precisa de um agente?
- ❑ Quando um único prompt é suficiente?
- ❑ Como evitar complexidade desnecessária?
- ❑ Em qual momento um workflow se torna um sistema multiagente?

LLMs como Componentes de Workflow

LLMs como Componentes de Workflow

- ❑ O modelo de linguagem não precisa ser utilizado apenas para gerar respostas finais
- ❑ Em arquiteturas baseadas em grafos, ele pode assumir diferentes papéis ao longo da execução
- ❑ **Exemplos:**
 - ❑ Classificar intenções
 - ❑ Extrair informações
 - ❑ Avaliar resultados
 - ❑ Tomar decisões
 - ❑ Gerar respostas



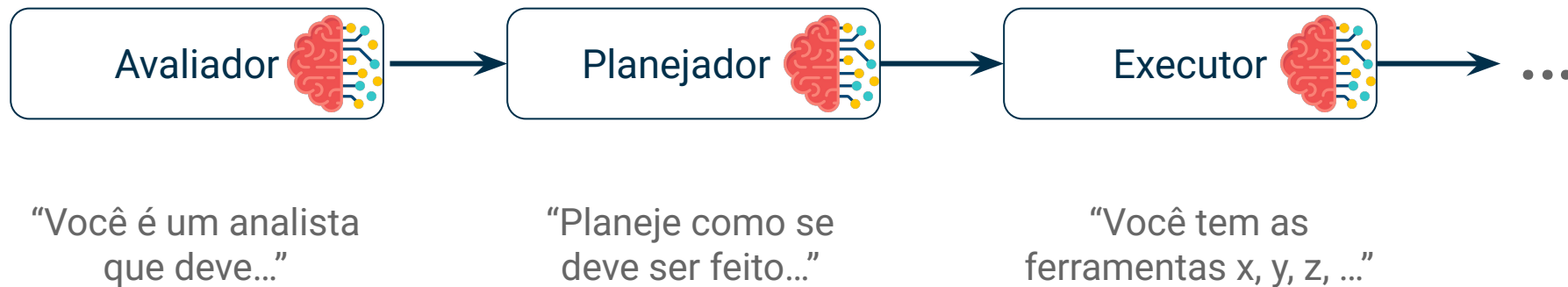
Especialização de Nós

- ❑ Diferentes nós podem ser especializados para tarefas específicas
- ❑ Problema complexo decomposto



Especialização de Nós via Prompts Específicos

- ❑ Ocorre por meio de diferentes prompts de sistema
 - ❑ Cada nó recebe uma responsabilidade clara dentro do workflow



LLM com diferentes Responsabilidades

- ❑ O LLM deixa de ser a aplicação inteira
- ❑ Ele passa a ser um componente dentro de uma arquitetura
- ❑ Cada LLM executa uma responsabilidade

Componente	Responsabilidade
Planejador	Define etapas
<i>Router</i>	Decide caminhos
Gerador	Produz conteúdo
Revisor	Avalia qualidade
Supervisor	Coordena

Benefícios

- ❏ Modularidade
- ❏ Reutilização
- ❏ Clareza arquitetural

Estado Conversacional

Estado Conversacional



Memória Compartilhada

O estado é a memória compartilhada entre os nós do sistema.



Objeto Vivo

Um objeto “vivo” que evolui durante a execução.

```
state = {  
  "objetivo": "...",  
  "plano": [],  
  "texto": "",  
  "feedback": "",  
  "iteracao": 0  
}
```

Estado Conversacional

- ❏ Em aplicações conversacionais, parte importante do estado é composta pelo **histórico** de mensagens.

Estado Conversacional

- ❑ Em aplicações conversacionais, parte importante do estado é composta pelo **histórico** de mensagens.
- ❑ O agente utiliza esse histórico para manter contexto entre diferentes etapas da execução.

Estado Conversacional

- ❑ Em aplicações conversacionais, parte importante do estado é composta pelo **histórico** de mensagens.
- ❑ O agente utiliza esse histórico para manter contexto entre diferentes etapas da execução.
- ❑ **Exemplos:**
 - ❑ Mensagens do usuário
 - ❑ Respostas do sistema
 - ❑ Resultados intermediários
 - ❑ Observações

Roteamento orientado por LLM

Roteamento orientado por LLM

- ❑ LLM pode atuar como um roteador
- ❑ Interpreta o estado atual
- ❑ Seleciona qual caminho do grafo deve ser seguido
 - ❑ Toma decisão da rota de execução do grafo

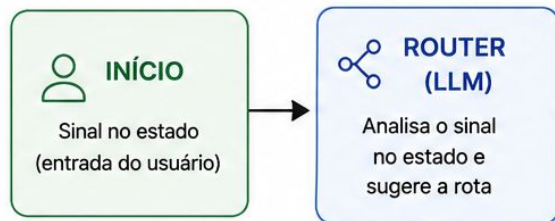
Roteamento orientado por LLM

- ❑ Roteamento deve ser controlado por categorias conhecidas
- ❑ O designer define quais caminhos existem
- ❑ O LLM escolhe entre rotas possíveis

Exemplo de Roteamento orientado por LLM

Sinal no estado	Rota sugerida	Responsabilidade do nó
Dúvida conceitual	explicador	organizar uma explicação
Pedido prático	exemplificador	gerar exemplo guiado
Resposta incompleta	revisor	avaliar e solicitar melhoria
Falta de contexto	perguntar	coletar informação adicional

Exemplo de Roteamento orientado por LLM



Roteamento

O ROUTER (LLM) interpreta o sinal no estado e escolhe o nó mais adequado para responder.

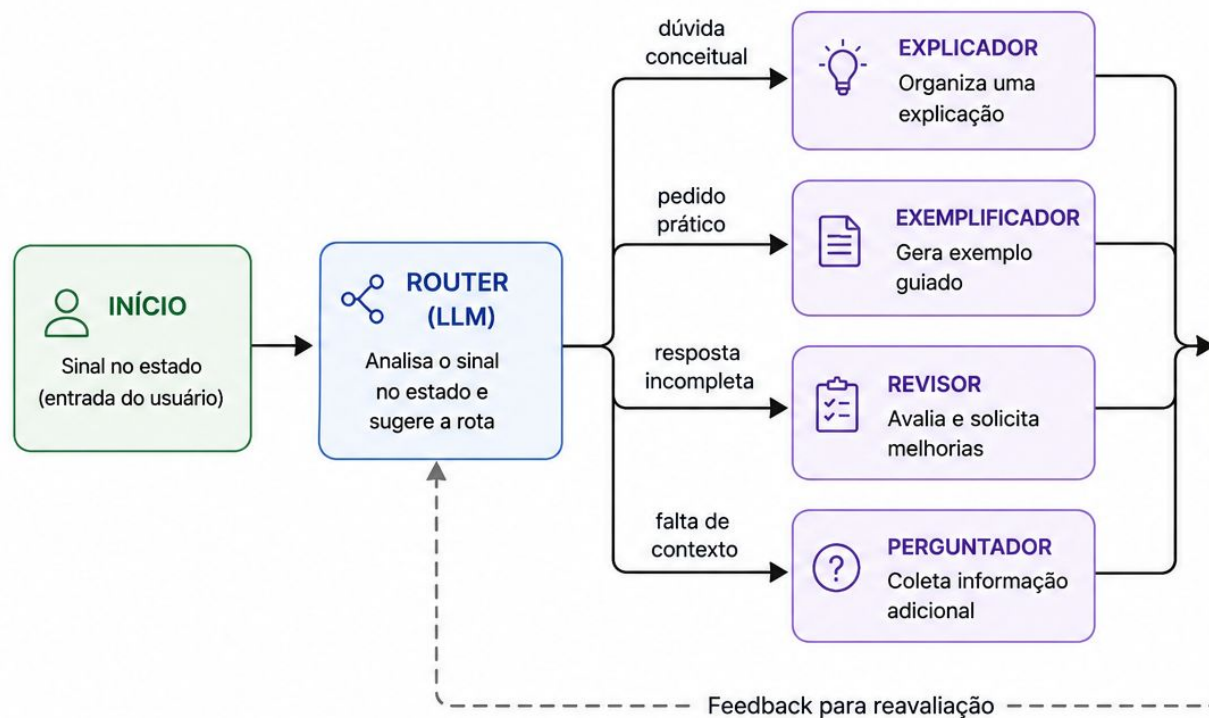
Nós especializados

Cada nó tem uma responsabilidade específica e contribui para resolver o problema do usuário.

Iteração

Se necessário, o feedback do Revisor ou do Agregador faz o fluxo retornar ao Router para nova decisão.

Exemplo de Roteamento orientado por LLM



Roteamento

O ROUTER (LLM) interpreta o sinal no estado e escolhe o nó mais adequado para responder.

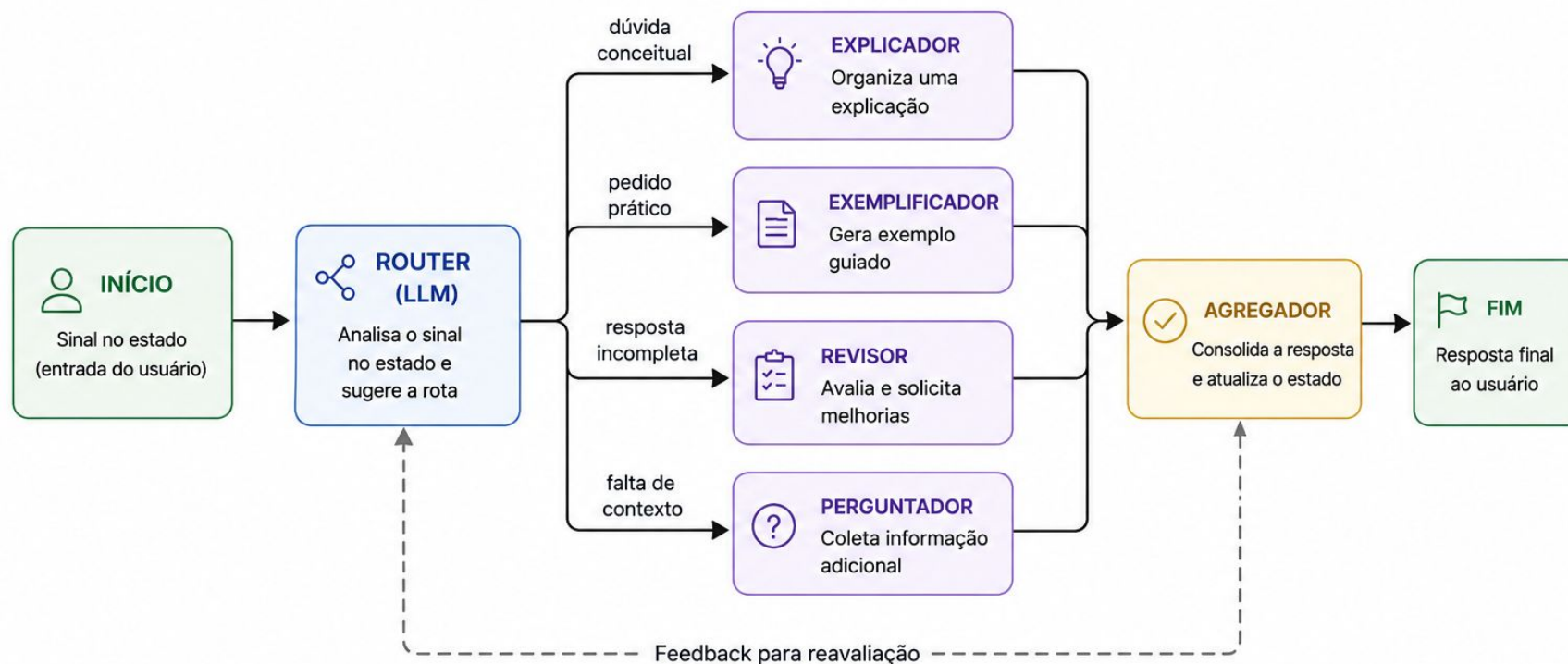
Nós especializados

Cada nó tem uma responsabilidade específica e contribui para resolver o problema do usuário.

Iteração

Se necessário, o feedback do Revisor ou do Agregador faz o fluxo retornar ao Router para nova decisão.

Exemplo de Roteamento orientado por LLM



Roteamento

O ROUTER (LLM) interpreta o sinal no estado e escolhe o nó mais adequado para responder.

Nós especializados

Cada nó tem uma responsabilidade específica e contribui para resolver o problema do usuário.

Iteração

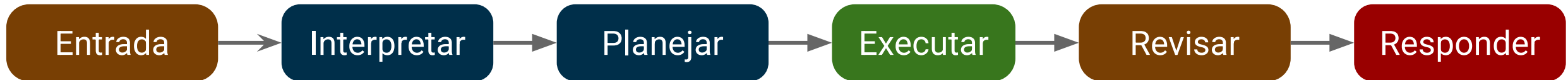
Se necessário, o feedback do Revisor ou do Agregador faz o fluxo retornar ao Router para nova decisão.

Workflow multi-etapas

Modelando um workflow completo

- ❏ Um workflow completo organiza a execução em etapas
- ❏ Passos bem definidos
 - ❏ Entende a demanda
 - ❏ Decide o plano
 - ❏ Executa ações
 - ❏ Revisa
 - ❏ Responde

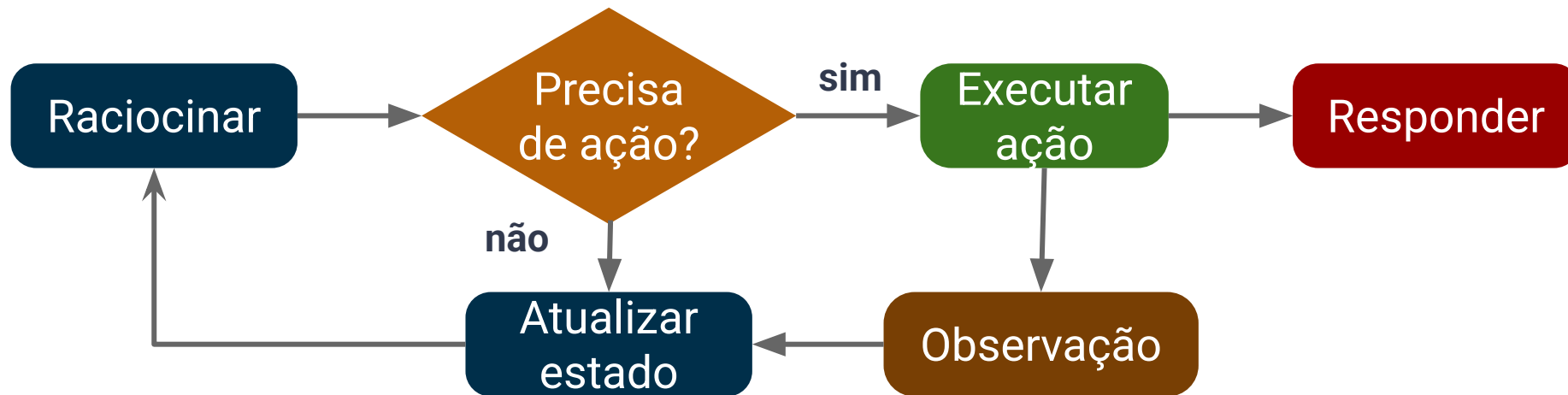
Modelando um workflow completo



(Cada etapa lê, produz e transmite atualizações no estado atual)

Raciocínio, ações e geração de resposta

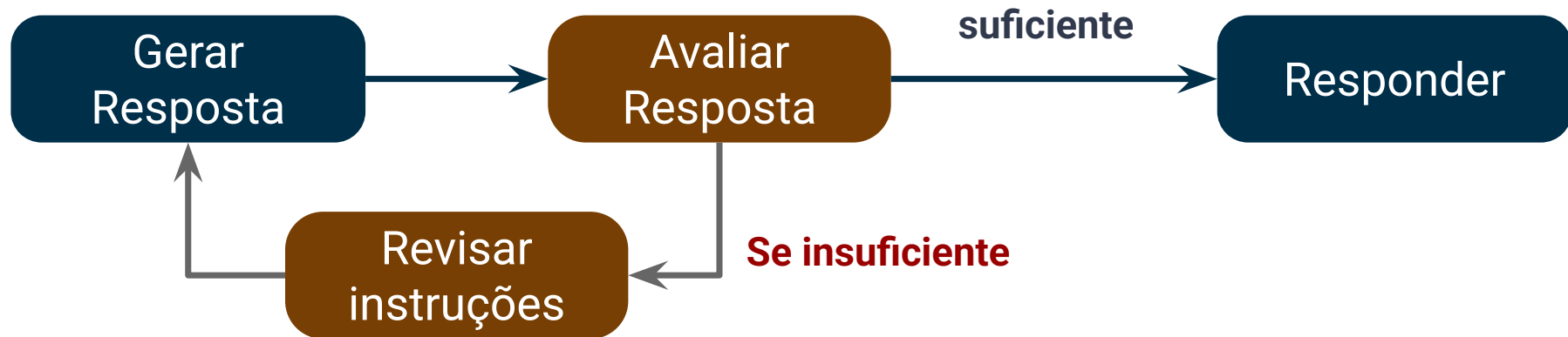
- ❑ Raciocínio e ação não precisam ficar em uma única chamada ao modelo



Loops de geração e revisão

- ❑ Agentes frequentemente precisam revisar, refinar ou corrigir uma saída antes de entregar a resposta final
- ❑ **Loops** tornam esse comportamento explícito

Loops de geração e revisão

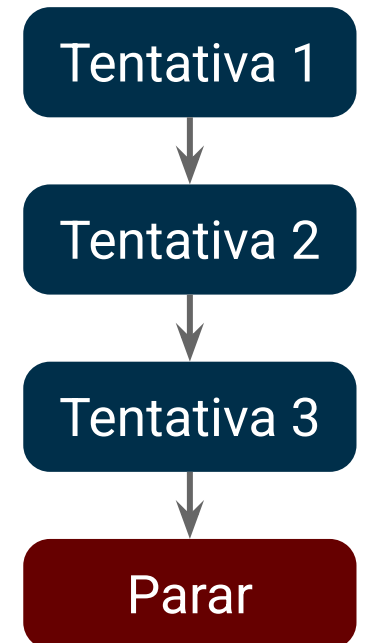


Controle de execução

- ❑ Loops precisam de critérios de parada
- ❑ Sem controle, um agente pode repetir indefinidamente
- ❑ ou gastar recursos sem melhorar a resposta

Controle de execução

- ❏ Limite máximo de iterações
- ❏ Critério mínimo de qualidade
- ❏ Detecção de ausência de melhoria
- ❏ Intervenção humana quando necessário



Generalizando Padrões de Arquitetura

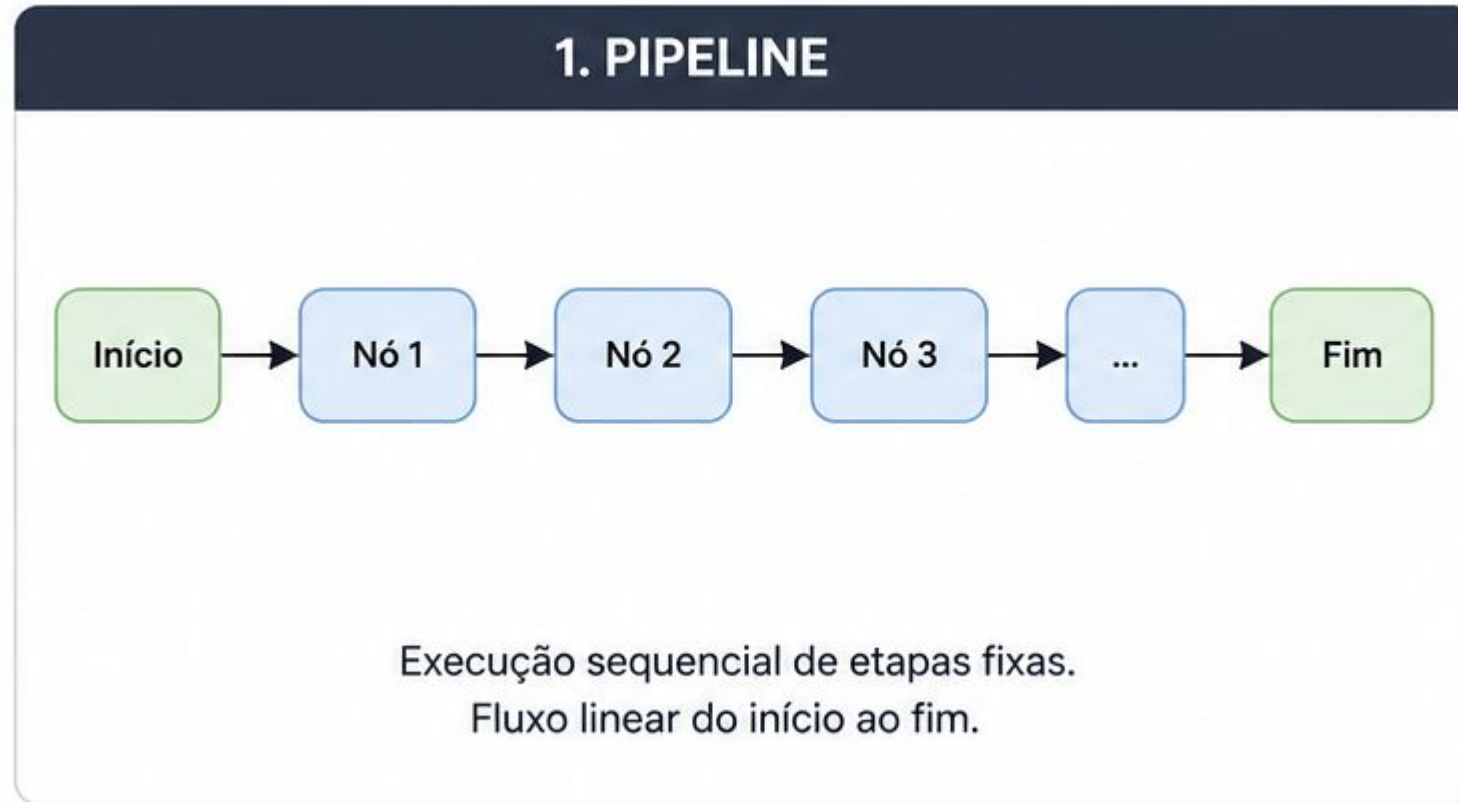
Padrões de arquitetura

- ❑ A mesma “base de grafos” permite diferentes padrões de agente
- ❑ A escolha depende do grau de controle, da complexidade da tarefa e da necessidade de revisão

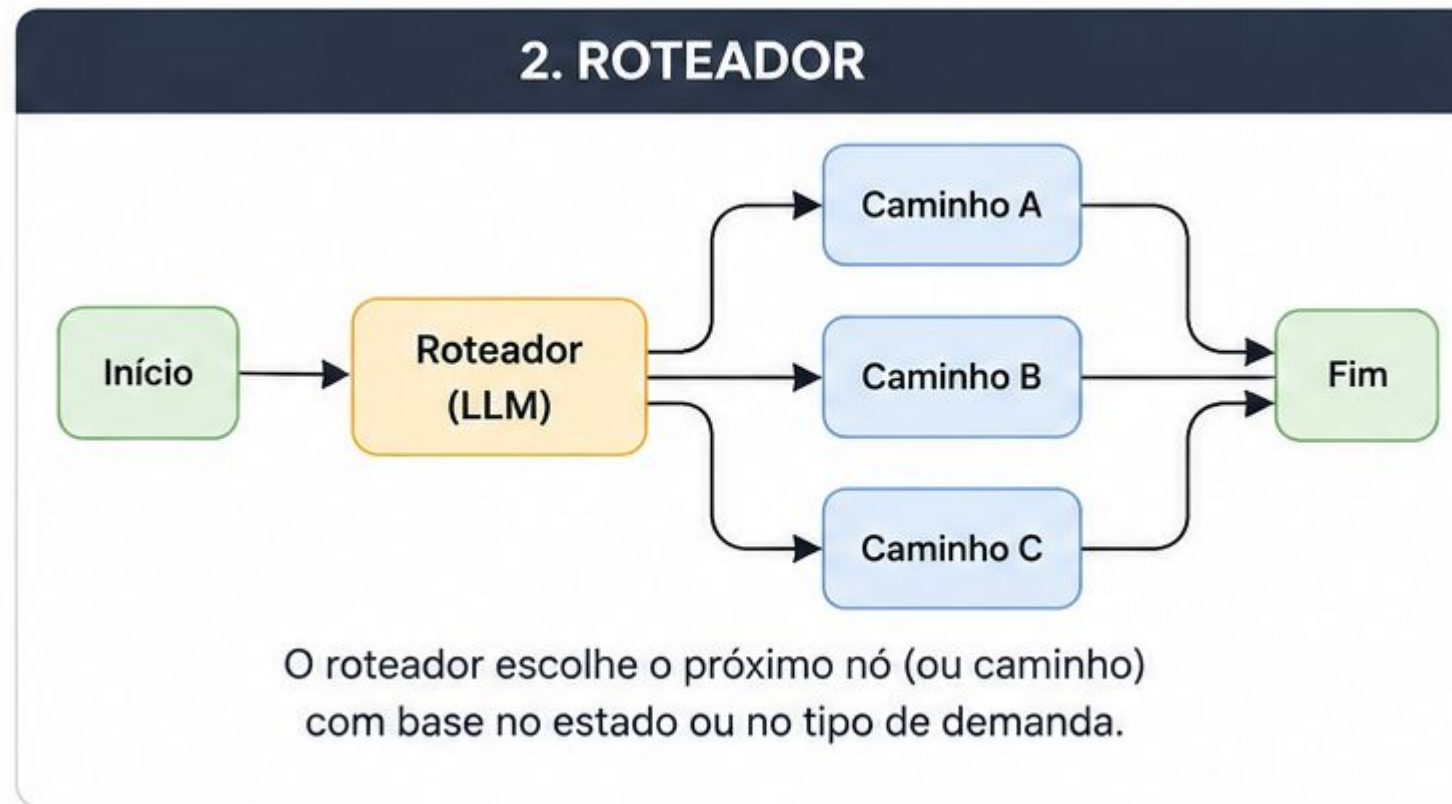
Padrões de arquitetura

Padrão	Racional	Quando usar
Pipeline	etapas fixas	tarefas previsíveis
Roteador	caminhos escolhidos via o estado	múltiplos tipos de demanda
Reflexivo	gerar, avaliar e revisar	qualidade é crítica
Supervisor	nó coordena especialistas	tarefas compostas

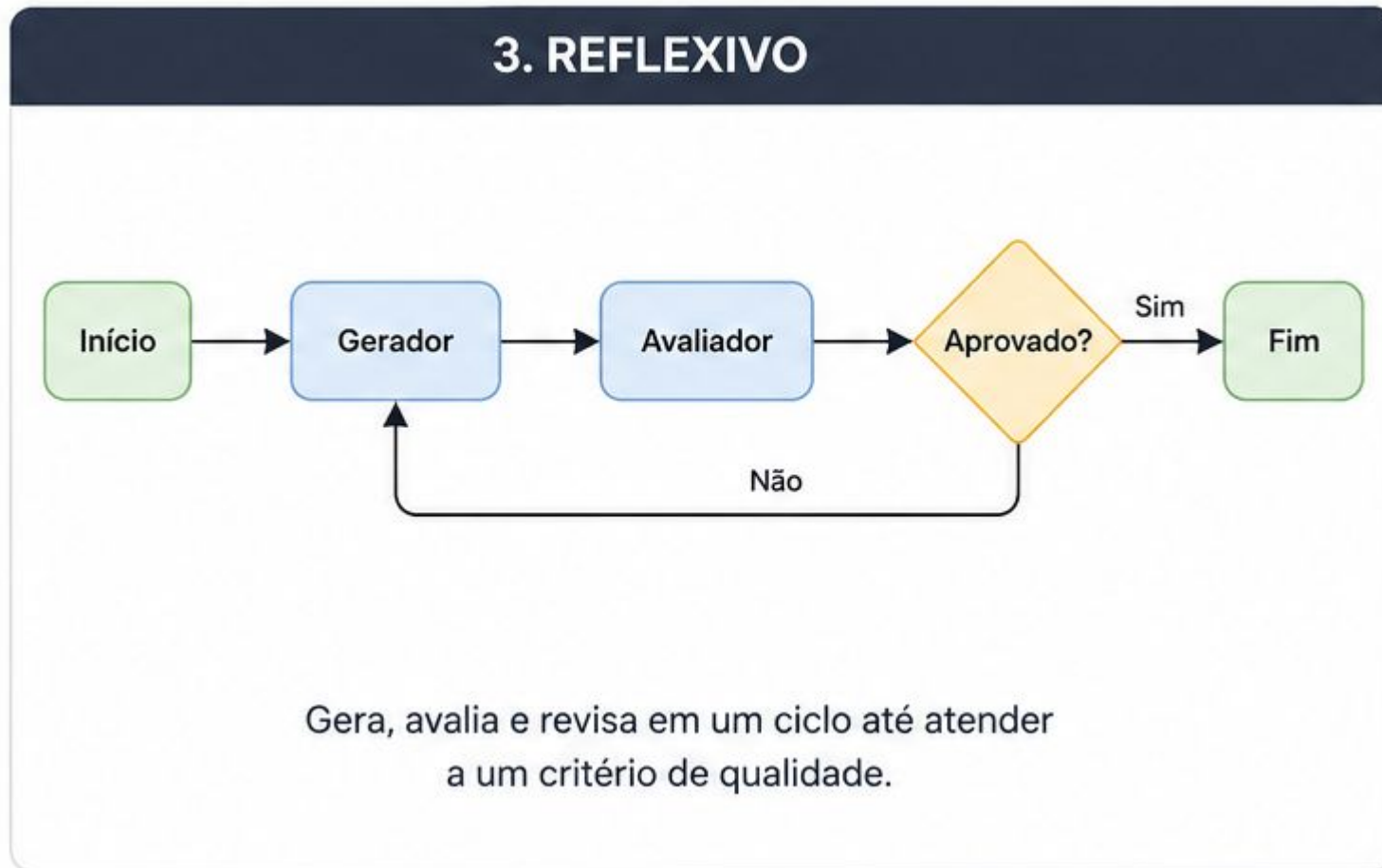
Pipeline



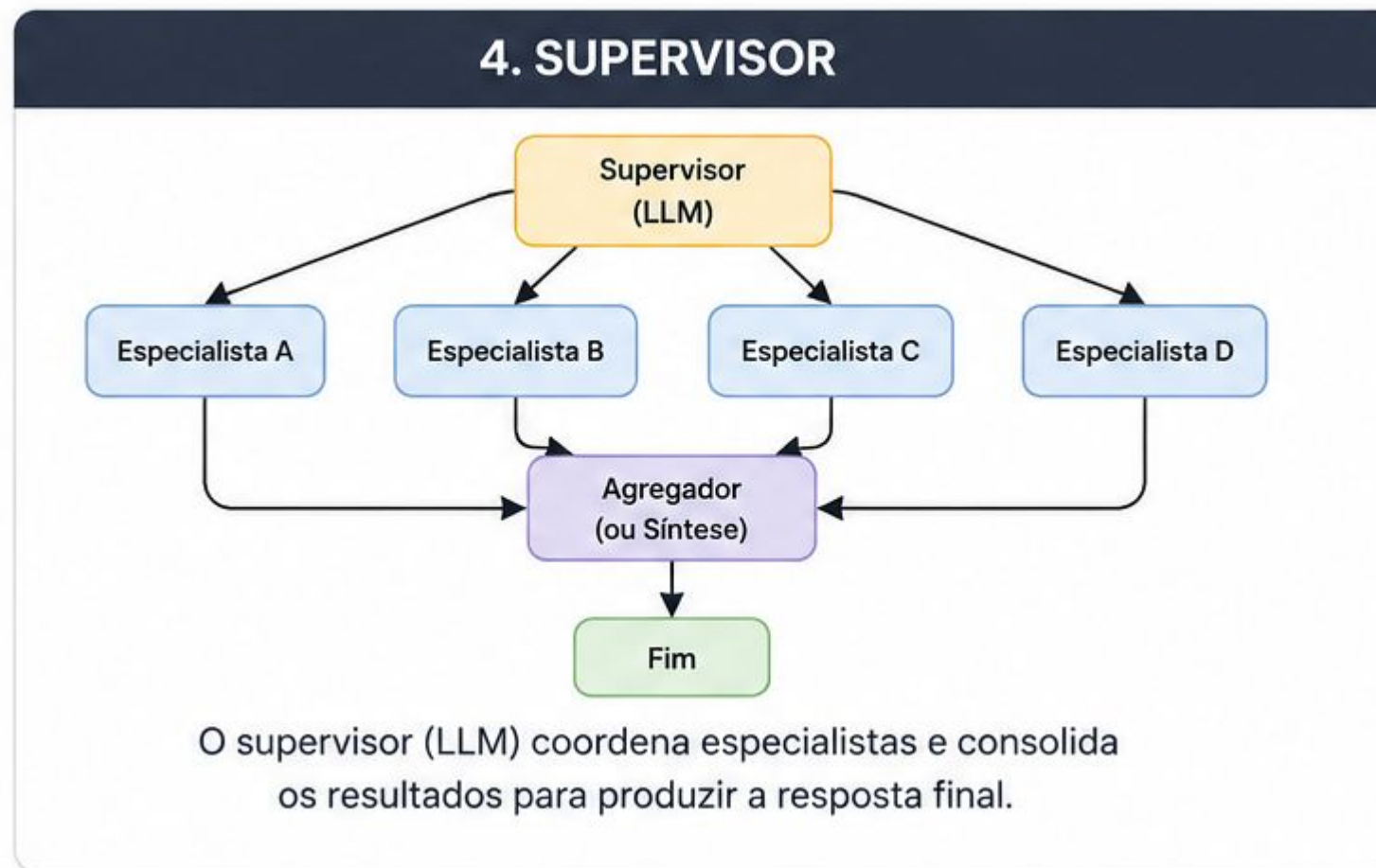
Roteador



Reflexivo



Supervisor

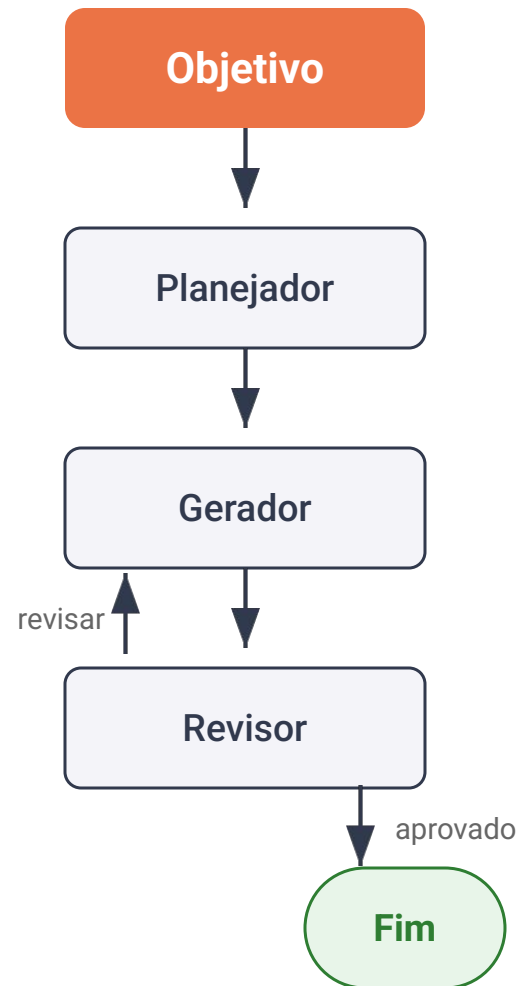


Exemplo

Geração de relatório

- ❏ Gerar um relatório utilizando planejamento, geração e revisão

Geração de relatório



Este fluxo representa um **Padrão de Arquitetura** para agentes autônomos, focando na iteração entre planejamento, geração e revisão para garantir a qualidade da resposta final.

Geração de relatório

Implementação de Agente Autônomo com LangGraph

```
from typing import TypedDict
from langgraph.graph import StateGraph, END

class State(TypedDict):
    objetivo: str
    plano: str
    texto: str
    feedback: str
    iteracao: int

def planejador(state):
    state["plano"] = f"Plano para: {state['objetivo']}"
    return state

def gerador(state):
    state["texto"] = f"Objetivo: {state['objetivo']}..."
    return state
```

```
def revisor(state):
    if state["iteracao"] == 0:
        state["feedback"] = "Adicionar detalhes"
    else:
        state["feedback"] = "aprovado"
    state["iteracao"] += 1
    return state

def router(state):
    if state["feedback"] == "aprovado":
        return END
    return "gerador"
```

Geração de relatório

Configuração do Grafo e Execução

```
builder = StateGraph(State)
builder.add_node("planejador", planejador)
builder.add_node("gerador", gerador)
builder.add_node("revisor", revisor)
builder.set_entry_point("planejador")
builder.add_edge("planejador", "gerador")
builder.add_edge("gerador", "revisor")
builder.add_conditional_edges(
    "revisor",
    router
)
```

```
app = builder.compile()
resultado = app.invoke(
    {
        "objetivo": "Criar relatório",
        "plano": "",
        "texto": "",
        "feedback": "",
        "iteracao": 0
    }
)
print(resultado)
```

Síntese

- ❑ LangGraph permite projetar comportamentos complexos por meio de workflows explícitos
- ❑ O estado compartilhado conecta todo o ciclo de execução do agente
- ❑ Roteamentos condicionais transformam decisões em elementos arquiteturais observáveis
- ❑ Loops iterativos introduzem mecanismos de autoavaliação e refinamento.
- ❑ Arquiteturas em grafos separam raciocínio, ferramentas, revisão e geração de resposta

Referências

- ❏ [Documentação oficial do LangGraph](#)
- ❏ [LangGraph – Graph API:](#)
- ❏ [Repositório oficial do LangGraph](#)
- ❏ [Huggingface: Introdução ao LangGraph](#)